2. REQUIREMENT ANALYSIS

EduGenie — Google Gemini Powered Learning Assistant

Functional, non-functional, system, interface, and technology requirements

Project Overview

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. It is designed for students of different academic levels and supports smart question answering, simplified explanations, quiz generation, personalized learning recommendations, and summarization of large educational passages.

Functional Requirements

ID	Requirement
FR-01	Accept educational questions or text from the learner through the web interface.
FR-02	Provide concise answers for general knowledge and academic questions.
FR-03	Explain complex concepts in simplified, readable language.
FR-04	Generate three multiple-choice questions from a supplied passage, with four options for each question.
FR-05	Summarize long educational passages while retaining the core information.
FR-06	Generate a structured learning path from beginner to advanced topics.
FR-07	Provide learning recommendations with useful resources such as videos, articles, or books when generated.
FR-08	Provide a task selector for Explain, QnA, Quiz, Summary, and Recommend Path.
FR-09	Send form submissions to the FastAPI backend and display the generated result below the input area.
FR-10	Expose RESTful API endpoints for each major learning module.
FR-11	Support local execution through Uvicorn for development and functional testing.

Non-Functional Requirements

Area	Requirement
Usability	Simple HTML + CSS interface with clear task selection, text input, submit action, and readable results.
Performance	Use a lightweight local explanation model and cloud-based Gemini services for appropriate tasks.
Maintainability	Keep the five learning functions separated into individual Python modules.
Scalability	Use a modular FastAPI backend so additional educational functions can be integrated later.
Accessibility	Keep explanations concise and easy to understand for beginners and self-learners.
Reliability	Handle AI response or parsing errors and return a useful error message.
Compatibility	Project documentation specifies Python 3.10+ and local execution with Uvicorn.

System Requirements and Dependencies

The project specifies Python 3.10+, FastAPI, Uvicorn, HTML/CSS, Jinja2, and a Google Gemini API key. Gemini 1.5 Pro is used for Q&A;, summarization, quiz generation, and learning paths, while LaMini-Flan-T5-783M is used for concept explanation.

Data / Input Requirements

The learner provides a question, topic, or educational passage. Depending on the selected task, the system transforms this input into an answer, explanation, quiz, summary, or structured learning path. The quiz module expects a structured

JSON response and cleans Markdown code blocks before parsing.

3. PROJECT DESIGN

EduGenie — Architecture, components, modules, and interface design

System Architecture

The architecture follows a lightweight web pattern: the learner interacts with the HTML/CSS frontend, the form sends a request to the FastAPI backend, and FastAPI routes the request to the selected AI module. Gemini 1.5 Pro handles cloud-based educational generation, while LaMini-Flan-T5-783M handles simplified concept explanations.

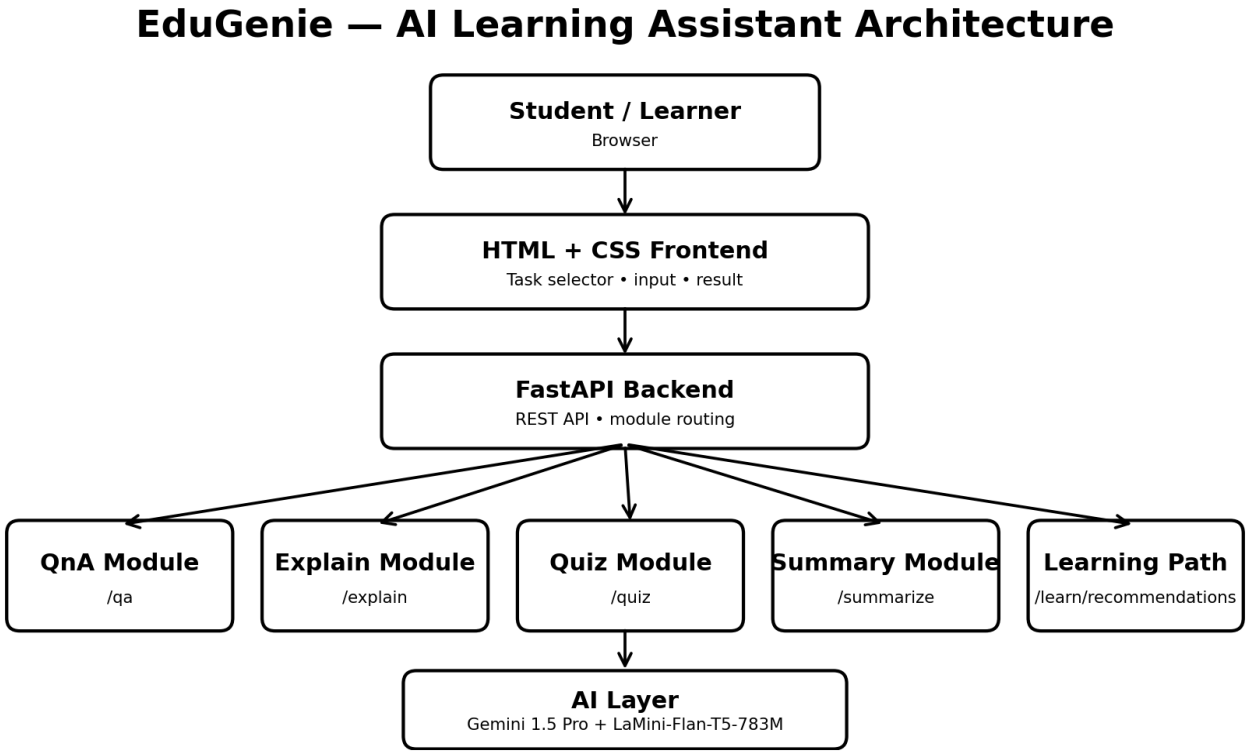


Fig. EduGenie Architecture Diagram

Component / Module Design

Module	Main responsibility
main.py	FastAPI application entry point.
explanation_module.py	Uses LaMini-Flan-T5-783M to explain concepts in simple, concise language.
qna.py	Uses Gemini 1.5 Pro for general knowledge and academic question answering.
quiz_module.py	Creates three MCQs with four options each and a correct answer in structured JSON.
summary_module.py	Summarizes long educational passages while preserving core information.
learning_path.py	Creates beginner-to-advanced learning recommendations and useful resources.
templates/index.html	Provides task selection, text input, submit control, and result display.
static/style.css	Provides responsive presentation and styling.
requirements.txt	Stores Python dependencies for the application.

API Design

Endpoint	Function
/qa	Question answering
/explain	Simplified concept explanation

Endpoint	Function
/quiz	MCQ generation
/summarize	Educational text summarization
/learn/recommendations	Personalized learning recommendations

Interface Design

The frontend contains a task dropdown with Explain, QnA, Quiz, Summary, and Recommend Path options, a text area for user input, a submit button, and a result container. The documentation specifies responsive styling and live integration in which the result appears below the input box.

Project Workflow / Milestones

Milestone 1 — Model Selection and Architecture: Gemini 1.5 Pro is selected for Q&A;, summarization, quiz generation, and learning paths; LaMini-Flan-T5-783M is selected for concept explanation. The project folder architecture separates each module.

Milestone 2 — Core Functionalities: Implement the explanation, QnA, quiz, summary, and learning-path modules, then expose them through FastAPI REST endpoints.

Milestone 3 — Frontend Development: Build the HTML form and CSS styling, then connect form submission to the FastAPI backend.

Milestone 4 — Deployment and Testing: Run the project using **uvicorn main:app --reload**, open **http://127.0.0.1:8000**, and test questions, explanations, quizzes, summaries, and learning plans.

User Scenarios

1. A student asks EduGenie a question such as which is the largest ocean and receives an answer. 2. A student selects Generate Quiz to check understanding of the Pythagoras Theorem. 3. A learner exploring SQL requests a structured beginner-to-advanced learning path with timelines and resources.

Quick Reference

Layer	Technology	Responsibility
Frontend	HTML + CSS	User input and result presentation
Backend	FastAPI	REST API and request routing
AI — Cloud	Gemini 1.5 Pro	Q&A, quiz, summary, learning path
AI — Local	LaMini-Flan-T5-783M	Concept explanation
Server	Uvicorn	Local ASGI application server
Templating	Jinja2	HTML templating
Language	Python 3.10+	Application development